

EXERCÍCIOS - POLIMORFISMO

INSTRUÇÕES GERAIS:

- Todos os exercícios devem usar Scanner para entrada de dados
- Crie menu interativo com do-while
- Use ArrayList para armazenar objetos
- Demonstre POLIMORFISMO (uma variável do tipo pai, objetos de tipos filhos)
- Separe as classes em arquivos diferentes (.java)

1: SISTEMA DE TRANSPORTES

Uma empresa de logística precisa calcular o custo de diferentes tipos de transporte.

Classe: Transporte

- Atributos: distancia (double), passageiros (int)
- Construtor
- Método: double calcularCusto()
return 0.0;
- Método: void exibirDetalhes()
- Getters

Subclasse: Carro extends Transporte

- Atributo adicional: tipoCombustivel (String)
- Implementar calcularCusto(): distancia * 1.50 (R\$ 1,50 por km)
- Implementar exibirDetalhes()

Subclasse: Onibus extends Transporte

- Atributo adicional: numeroLinhas (int)
- Implementar calcularCusto(): distancia * 0.80 (R\$ 0,80 por km)
- Implementar exibirDetalhes()

Subclasse: Bicicleta extends Transporte

- Atributo adicional: tipoFreio (String)
- Implementar calcularCusto(): 0.0 (gratuito!)
- Implementar exibirDetalhes()

MAIN COM MENU:

1. Adicionar Carro
2. Adicionar Ônibus
3. Adicionar Bicicleta
4. Listar todos os transportes
5. Calcular custo total de todos
0. Sair

EXEMPLO DE SAÍDA:

==== CARRO ====

Distância: 100 km

Passageiros: 4

Combustível: Gasolina

Custo: R\$ 150,00

2: SISTEMA DE ANIMAIS NO ZOOLOGICO

Um zoológico precisa gerenciar informações sobre diferentes tipos de animais.

Classe: Animal

- Atributos: nome (String), idade (int), peso (double)
- Construtor
- Método: void emitirSom()
- Método: String getTipoAlimento()
return "Alimento";
- Método: void exibirInfo()

Subclasse: Leao extends Animal

- Implementar emitirSom(): "ROARRR!"
- Implementar getTipoAlimento(): "Carnívoro - Carne"
- Método===== void rugir()

Subclasse: Elefante extends Animal

- Atributo adicional: comprimentoElefante (double)
- Implementar emitirSom(): "PRRRUUUUU!"
- Implementar getTipoAlimento(): "Herbívoro - Folhas e frutas"

Subclasse: Papagaio extends Animal

- Atributo adicional: corPenas (String)
- Implementar emitirSom(): "Olá! Quero biscoito!"
- Implementar getTipoAlimento(): "Onívoro - Sementes e frutas"

- Método: void falar(String frase)

MAIN COM MENU:

1. Adicionar Leão
2. Adicionar Elefante
3. Adicionar Papagaio
4. Listar todos os animais
5. Fazer todos os animais emitirem som
6. Mostrar dieta de todos
0. Sair

EXEMPLO DE INTERAÇÃO:

Escolha: 5

== ANIMAIS EMITINDO SOM ==

Rex diz: ROARRR!

Dumbo diz: PRRRUUUUU!

Loro diz: Olá! Quero biscoito!

3: SISTEMA DE FUNCIONÁRIOS

Uma empresa precisa calcular salários de diferentes tipos de funcionários.

Classe: Funcionario

- Atributos: nome (String), salarioBase (double)
- Construtor
- Método: double calcularSalario()
return 0.0;
- Método: void exibirContracheque()
- Getters e Setters

Subclasse: Gerente extends Funcionario

- Atributo adicional: bonusFixo (double)
- Implementar calcularSalario(): salarioBase + bonusFixo
- Implementar exibirContracheque()

Subclasse: Vendedor extends Funcionario

- Atributos adicionais: totalVendas (double), comissao (double, em %)
- Implementar calcularSalario(): salarioBase + (totalVendas * comissao)
- Implementar exibirContracheque()

Subclasse: Programador extends Funcionario

- Atributos adicionais: horasExtras (int), valorHoraExtra (double)
- Implementar calcularSalario(): salarioBase + (horasExtras * valorHoraExtra)

- Implementar `exibirContracheque()`

MAIN COM MENU:

1. Cadastrar Gerente
2. Cadastrar Vendedor
3. Cadastrar Programador
4. Listar todos os contracheques
5. Calcular folha de pagamento total
6. Buscar funcionário por nome
0. Sair

EXEMPLO DE SAÍDA:

==== CONTRACHEQUE - VENDEDOR ====

Nome: João Silva

Salário Base: R\$ 2.000,00

Total Vendas: R\$ 10.000,00

Comissão (5%): R\$ 500,00

SALÁRIO TOTAL: R\$ 2.500,00